

# Predicting Audience Movie Ratings from TMDb Metadata

JSC370 Final Project, Winter 2026

Yujun Lu

## 1 Introduction

Online movie databases like TMDb collect a lot of metadata for every film: ratings, vote counts, popularity scores, runtime, genre, budget, revenue. On the surface these variables look like they should explain audience ratings, but my midterm exploratory analysis on 148 movies showed otherwise. Popularity was almost uncorrelated with rating. Budget and revenue were strongly tied to each other but barely related to rating. Genre medians differed but the distributions overlapped a lot. Nothing obvious popped out as a strong rating predictor on its own.

That left an open question for the final: if no single variable explains ratings, can a model that combines all of them do better? This report takes the same TMDb data source and asks it as a prediction problem.

I focus on two questions:

1. *Regression.* How accurately can I predict TMDb's `vote_average` from a film's metadata using methods covered in JSC370?
2. *Classification.* Using the same features, can I tell whether a film will be high-rated (rating at least 7.0) on a held-out test set?

For both, I compare a linear baseline against Random Forest and XGBoost and look at which features the trees rely on most.

## 2 Methods

### 2.1 Data collection

All data comes from The Movie Database public REST API.<sup>1</sup> The midterm used the `/movie/popular` endpoint, which kept returning the same currently-trending titles and gave me only 148 usable rows after I filtered out movies with too few votes. For the final, I switched to `/discover/movie` and combined five sort strategies to get a wider sample: most popular post-2015, most popular pre-2015, most-voted overall, top-rated, and highest grossing. After deduplicating on TMDb's `id` and filtering to movies with at least 30 votes, the final dataset has roughly 1,500 films. Per-movie details (budget, runtime, genres, production companies, etc.) come from `/movie/{id}`. The collection script (`collect_data.qmd`) is fully reproducible.

---

<sup>1</sup><https://developer.themoviedb.org/reference/intro/getting-started>

## 2.2 Cleaning and feature engineering

Numeric fields were coerced to floats. TMDb stores 0 for unknown budget, revenue, and runtime, so I recoded those zeros as missing. Following the midterm convention I dropped films with fewer than 30 votes, since their average ratings are too noisy to model.

Engagement and financial variables span several orders of magnitude, so I log-transformed `vote_count`, `popularity`, `budget`, and `revenue`. The `genres` field comes back from TMDb as a list of objects, which I expanded into 18 binary indicators (one per common genre) plus a `primary_genre` categorical and a count of genre tags per film. Production countries were turned into a count plus a US-production flag. Two missing-value flags (`has_budget`, `has_revenue`) preserve the fact that a film has no reported financial data, since “budget unknown” is itself informative. Decade and release month came from `release_date`. Detailed code is in `analysis.qmd`. After feature engineering each film has roughly forty numeric and indicator features.

Table 1 and Table 2 report descriptive statistics for the cleaned dataset.

Table 1: Summary statistics for the main analysis dataset.

|        | vote_average | popularity | vote_count | runtime | release_year |
|--------|--------------|------------|------------|---------|--------------|
| count  | 1504         | 1504       | 1504       | 1504    | 1504         |
| mean   | 7.2          | 16.47      | 6941.15    | 117.82  | 2008.33      |
| median | 7.24         | 10.57      | 5526.5     | 115     | 2013         |
| std    | 0.74         | 30.22      | 6238.72    | 25.04   | 18.22        |
| min    | 4.13         | 0.01       | 50         | 4       | 1902         |
| max    | 8.72         | 596.41     | 39491      | 253     | 2026         |

Table 2: Summary statistics for the financial subset (films with positive budget *and* revenue).

|        | budget      | revenue     |
|--------|-------------|-------------|
| count  | 1273        | 1273        |
| mean   | 7.29891e+07 | 3.05538e+08 |
| median | 4.8e+07     | 1.89016e+08 |
| min    | 23          | 22          |
| max    | 4.899e+08   | 2.92371e+09 |

## 2.3 Modelling protocol

I split the data 60/20/20 into training, validation, and test sets, with `random_state=370` for reproducibility. The classification target is stratified across the three splits.

For **regression** I fit three models:

- Linear regression on median-imputed features. This is a parametric baseline that ignores feature interactions.
- Random forest with 500 trees, no maximum depth, and a minimum leaf size of 2. Bagging plus random feature subsetting at each split decorrelates the trees and lowers variance.

- XGBoost with 600 boosting rounds, learning rate 0.05, max depth 5, row and column subsampling at 0.8, and L2 regularisation  $\lambda = 1$ . I monitored the validation set during training. Boosting fits trees to the gradient of the loss, so it tends to capture interactions more efficiently than a plain forest at the same capacity.

I report  $R^2$ ,  $\text{RMSE} = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$ , and  $\text{MAE} = \frac{1}{n} \sum |y_i - \hat{y}_i|$  on the held-out test set.

For **classification** the target is `is_high_rated` =  $1\{\text{vote\_average} \geq 7.0\}$ . I fit logistic regression as a baseline, plus random forest and XGBoost classifiers. The forest uses `class_weight="balanced"` and XGBoost uses `scale_pos_weight` to correct for the imbalance between high and low ratings. I report accuracy, ROC AUC, and the test-set ROC curve (Figure 6).

## 3 Results

### 3.1 Exploratory recap

Figure 1 confirms the midterm finding that ratings cluster near 6.5 and rarely go below 5 or above 8. Figure 2 extends the midterm correlation matrix to the engineered numeric features. `log_vote_count` and `log_popularity` are weakly positively correlated with rating (around 0.2 to 0.3); `log_budget` and `log_revenue` remain strongly correlated with each other (around 0.7) but only weakly with rating. Figure 3 shows that median ratings differ by genre — animation, history, and documentary tend higher; horror tends lower — but within-genre spread is wide and overlaps a lot.

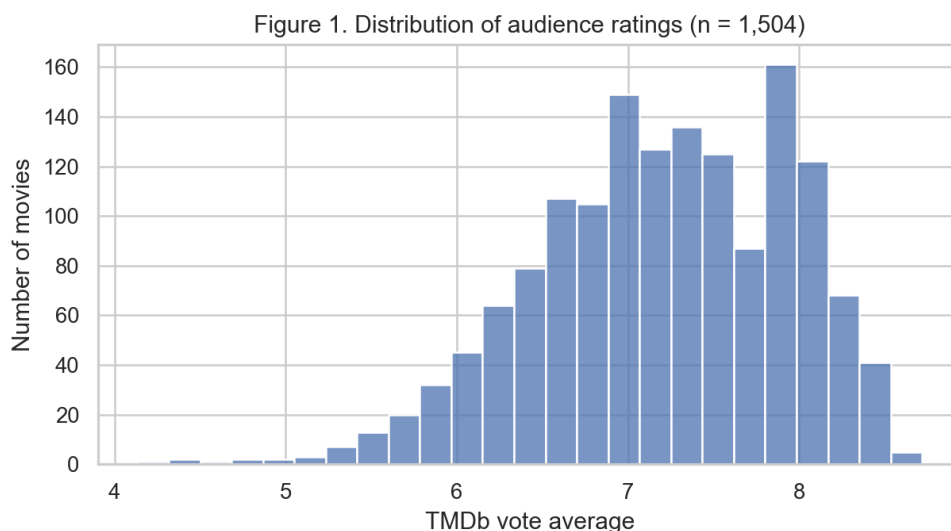


Figure 1: Distribution of audience ratings. Most films sit between 5.5 and 7.5, with the modal bin near 6.5.

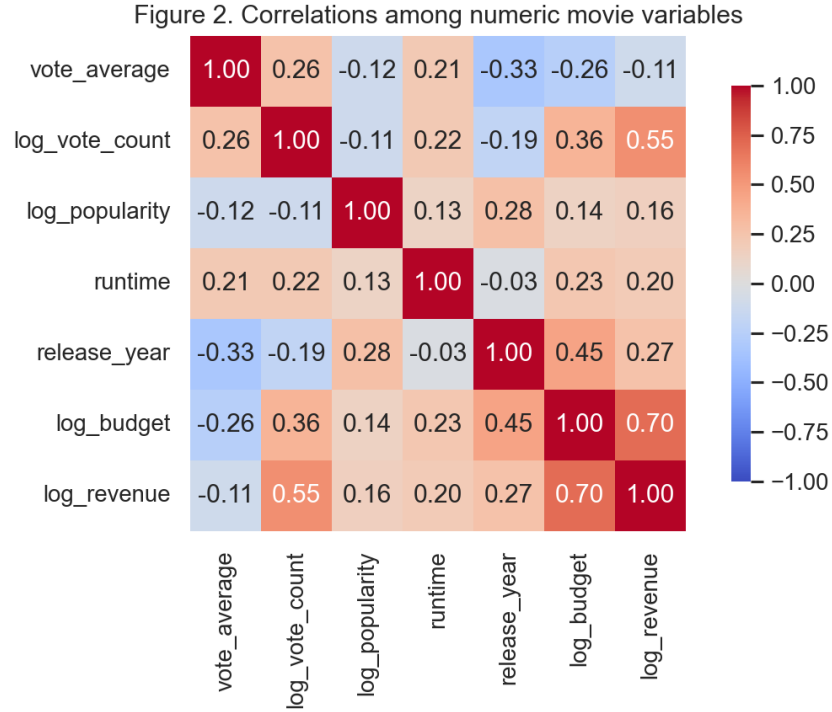


Figure 2: Correlation matrix of numeric movie variables. Log-transformed engagement and financial variables show modest positive associations with rating; budget and revenue correlate strongly with each other.

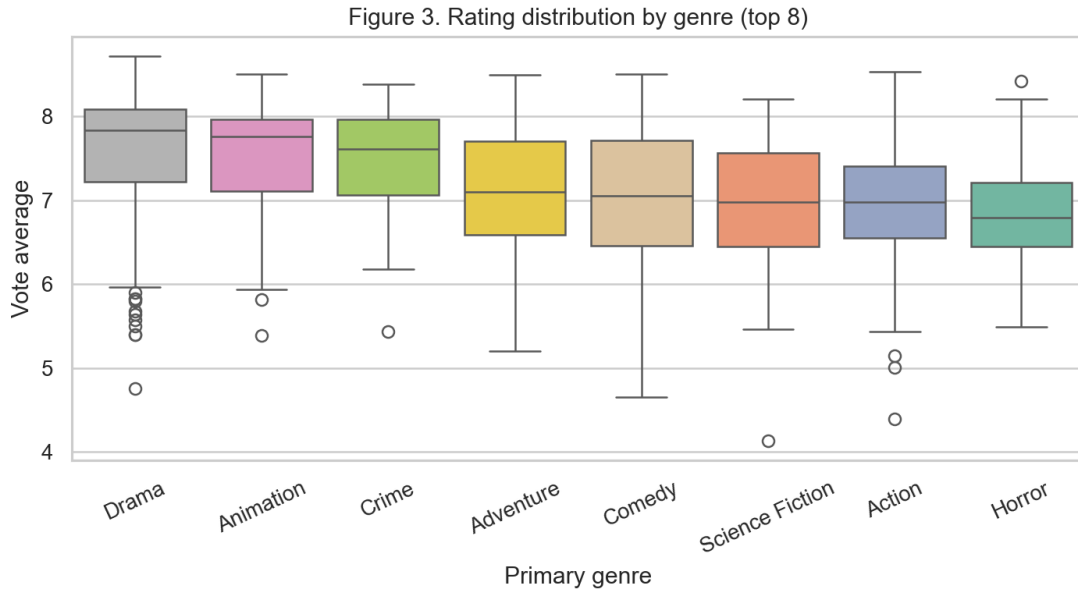


Figure 3: Audience ratings by genre (top 8 by frequency). Median ratings differ across genres but the distributions overlap heavily.

### 3.2 Regression performance

Table 3 compares the three regressors on the test set. Both tree-based models beat the linear baseline by a clear margin; XGBoost edges out random forest on every metric, but the gap between them is small (about one percentage point on  $R^2$  and 0.01 on RMSE). Figure 4 plots predicted versus actual ratings for each model. The linear baseline pulls predictions toward the mean (the shrinkage effect of squared-error linear regression). XGBoost’s predictions hug the 45-degree line more tightly, especially in the middle of the rating range. All three models tend to underpredict the very top of the rating scale, which is hard to fit with so few examples up there.

Table 3: Test-set regression metrics. Lower RMSE and MAE are better; higher  $R^2$  is better.

| model             | R2    | RMSE  | MAE   |
|-------------------|-------|-------|-------|
| Linear Regression | 0.374 | 0.597 | 0.448 |
| Random Forest     | 0.542 | 0.51  | 0.373 |
| XGBoost           | 0.557 | 0.502 | 0.366 |

Figure 4. Predicted vs actual ratings on the test set

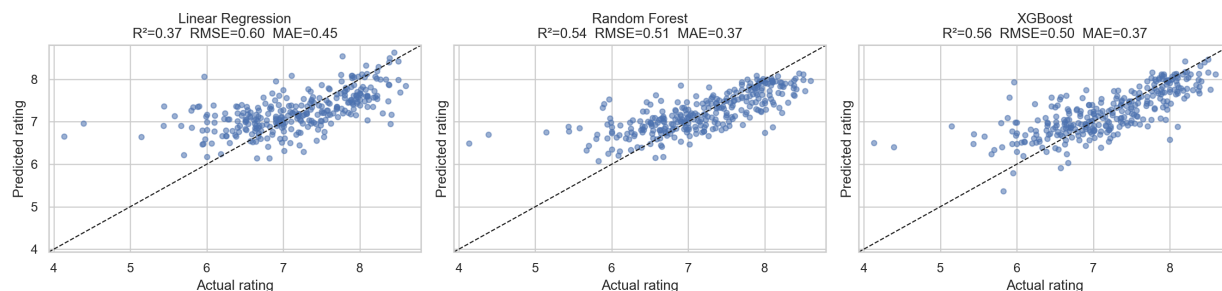


Figure 4: Predicted versus actual ratings on the test set, by model. The dashed black line is the identity. XGBoost shows the tightest scatter; the linear baseline is visibly biased toward the mean.

Figure 5 shows the top 15 features ranked by XGBoost gain. Categorical features dominate. `genre_animation` is by far the strongest predictor, followed by `genre_drama`, `is_english`, `release_year`, and `genre_music`. Several other genre indicators (`history`, `fantasy`, `action`, `horror`) and structural flags (`has_budget`, `belongs_to_collection`, `is_us_production`) follow close behind. Engagement features (`log_vote_count`, `log_popularity`) sit in the middle of the ranking rather than at the top. The midterm exploratory analysis already showed that median ratings differ noticeably across genres; the model is leaning on those genre and language differences, combined with release year, to separate higher-rated films from the rest.

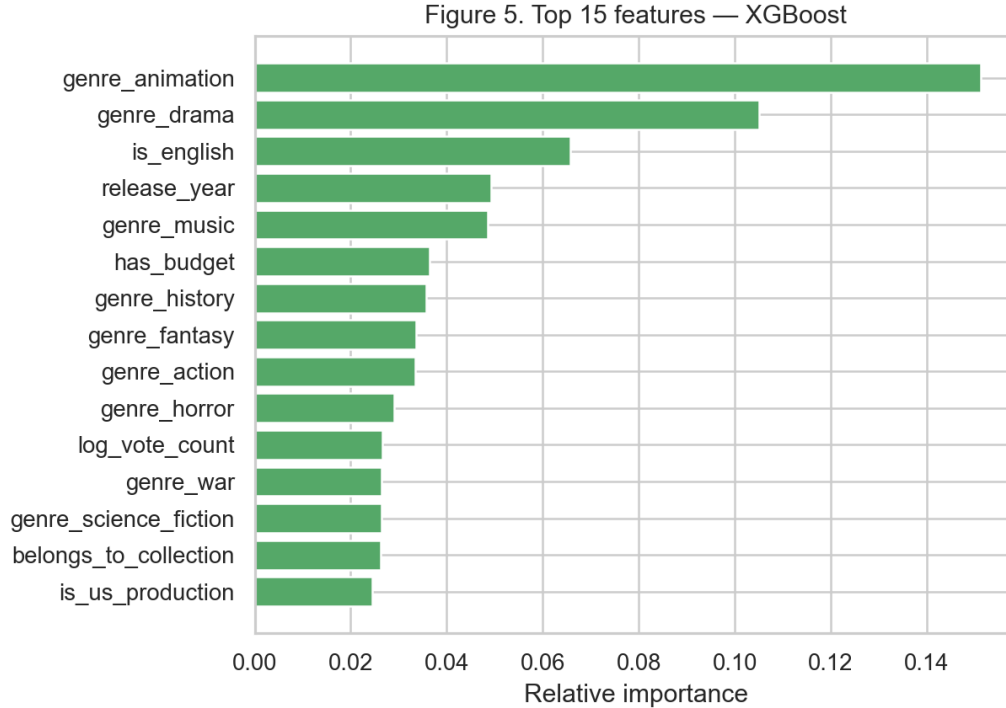


Figure 5: Top 15 features ranked by relative importance. Genre indicators (animation, drama, music) and language/rececity features dominate; engagement features sit lower in the ranking than the midterm analysis suggested they would.

### 3.3 Classification performance

Table 4 reports test-set accuracy and ROC AUC for the high-rating task. Tree-based models again come out ahead. XGBoost has the highest accuracy, while random forest has the highest ROC AUC, and the two are essentially tied — both sit in the mid-to-high 0.8s on AUC, well above the logistic baseline. Figure 6 plots the full ROC curves; the random forest and XGBoost lines are difficult to separate visually for most threshold ranges.

Table 4: Test-set classification metrics for predicting `vote_average` 7.0.

| model               | Accuracy | ROC_AUC |
|---------------------|----------|---------|
| Logistic Regression | 0.757    | 0.809   |
| Random Forest       | 0.787    | 0.876   |
| XGBoost             | 0.804    | 0.871   |

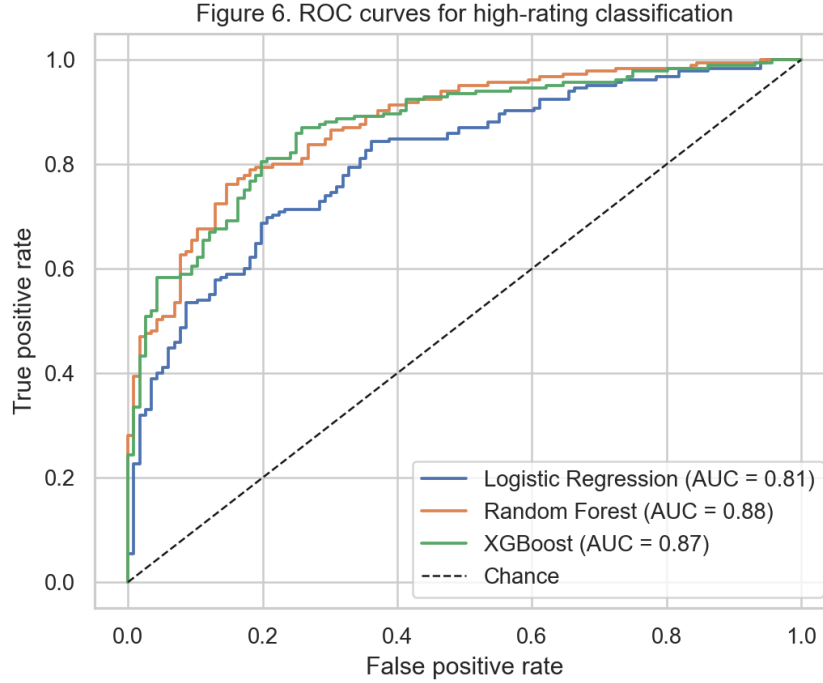


Figure 6: ROC curves for the high-rating classifier on the test set. The dashed grey line is chance.

## 4 Conclusions and Summary

So can audience ratings be predicted from public TMDb metadata? Yes, to a meaningful extent. Tree-based ensembles beat linear baselines on both the regression and classification versions of the question. XGBoost and random forest are roughly tied — XGBoost is marginally ahead on regression and classification accuracy, while random forest nudges ahead on ROC AUC. On the larger 1,500-film sample, the trees reach  $R^2 \approx 0.55$  on the continuous rating and ROC AUC near 0.87 on the high-rating task.

The story about *which* features carry the signal looks different from what the midterm hinted at. Genre indicators (animation, drama, music) and language and recency variables (`is_english`, `release_year`) sit at the top of the importance ranking, while engagement signals (`log_vote_count`, `log_popularity`) — which looked dominant in the smaller midterm sample — sit in the middle of the pack here. The expanded sample brings in more films from across genres, languages, and decades, and that diversity makes the genre/language structure of TMDb ratings more visible to the model.

A few caveats are worth flagging. Vote count is a post-release quantity, available at scoring time for any film already on TMDb but not before release, which is the more commercially interesting setting. A follow-up using only pre-release variables (cast, director, budget, studio, genre, planned release date) would sharpen the question. The dataset is also heavily skewed toward English-language and US productions, which inflates the apparent contribution of `is_english` and `is_us_production` and limits how well the model would generalise to international cinema. And TMDb ratings are themselves a self-selected sample — people who bother to rate a film are not a random sample of viewers.

None of these caveats invalidate the main finding: TMDb metadata contains real but limited rating signal, and tree ensembles use that signal more efficiently than linear models do. Full source code, the collection notebook, the analysis notebook, and the interactive figures are all in the project GitHub repo and on the website.